SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name of Student: Postiwala Vatsal Dipakbhai	PRN: 26070122509
Subject: Web Application and Development	Sem: III AY: 2026-27
EXPERIMENT No: 7	Assignment No: 7
TITLE: To implement a backend server-side web application using Node.js incorporating modular architecture patterns and the native network HTTP module.	

Objective

To install Node.js, understand its module system (built-in, local and npm modules), and build a basic multi-route web server using the core http module.

Theory

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run outside the browser, enabling server-side development. It uses a single-threaded, event-driven, non-blocking I/O model, making it efficient for I/O-heavy applications. Node's module system lets code be split into reusable files: built-in modules (http, fs, url, events, os, path) are available without installation; local modules are files you create and export via module.exports; npm modules are third-party packages installed via npm install. The http module's createServer() lets you build a raw web server that listens on a port and responds based on the request's method and URL.

Software / Hardware Requirements

Node.js LTS installed (verify with node -v and npm -v in terminal), VS Code.

Step-by-Step Procedure

1. Verify Node.js installation: open a terminal and run node -v and npm -v. If not installed, download the LTS installer from nodejs.org and install it.

2. Create a project folder lab7_node and initialize it as an npm project.

```
mkdir lab7_node && cd lab7_node
npm init -y
```

3. Create a file math.js as a local module exporting simple utility functions, to demonstrate the local module system.

```
// math.js
function add(a, b) { return a + b; }
function square(a) { return a * a; }
module.exports = { add, square };
```

4. Create index.js and require both a built-in module (os) and your local module (math.js) to print

some info to the console, confirming the module system works.

```
// index.js
const os = require('os');
const math = require('./math');

console.log('Platform:', os.platform());
console.log('CPU Cores:', os.cpus().length);
console.log('2 + 3 =', math.add(2, 3));
console.log('4 squared =', math.square(4));
```

5. Run node index.js and verify console output shows platform info and the math results.

6. Now build the HTTP server. Require the built-in http module and use http.createServer() with a request listener that inspects req.url.

```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'text/html');

  if (req.url === '/') {
    res.statusCode = 200;
    res.end('<h1>Home Page</h1><p>Welcome to my Node.js server</p>');
  } else if (req.url === '/about') {
    res.statusCode = 200;
    res.end('<h1>About Page</h1><p>This server demonstrates the HTTP
module.</p>');
  } else if (req.url === '/contact') {
    res.statusCode = 200;
    res.end('<h1>Contact Page</h1><p>Email: student@example.com</p>');
  } else {
    res.statusCode = 404;
    res.end('<h1>404 - Page Not Found</h1>');
  }
});

const PORT = 3000;
server.listen(PORT, () => console.log('Server running at http://localhost:' +
PORT));
```

7. Run the server with node server.js and, in a browser, visit http://localhost:3000/, /about, /contact and a non-existent route (e.g. /xyz) to confirm each responds correctly, including the 404 handler.

8. Modify the server to differentiate HTTP methods: check req.method so a GET on / returns the home page while any other method returns a 405 'Method Not Allowed' status.

```
if (req.url === '/' && req.method === 'GET') {
  res.statusCode = 200;
```

```
  res.end('<h1>Home Page (GET)</h1>');
} else if (req.method !== 'GET') {
  res.statusCode = 405;
  res.end('Method Not Allowed');
}
```

9. Stop the server (Ctrl+C) and add nodemon as a dev dependency to auto-restart the server on file changes; update package.json's scripts.

```
npm install --save-dev nodemon
```

```
"scripts": { "start": "node server.js", "dev": "nodemon server.js" }
```

10. Run npm run dev, edit a route's message in server.js, save, and observe nodemon automatically restart the server without manually re-running node.

Output:

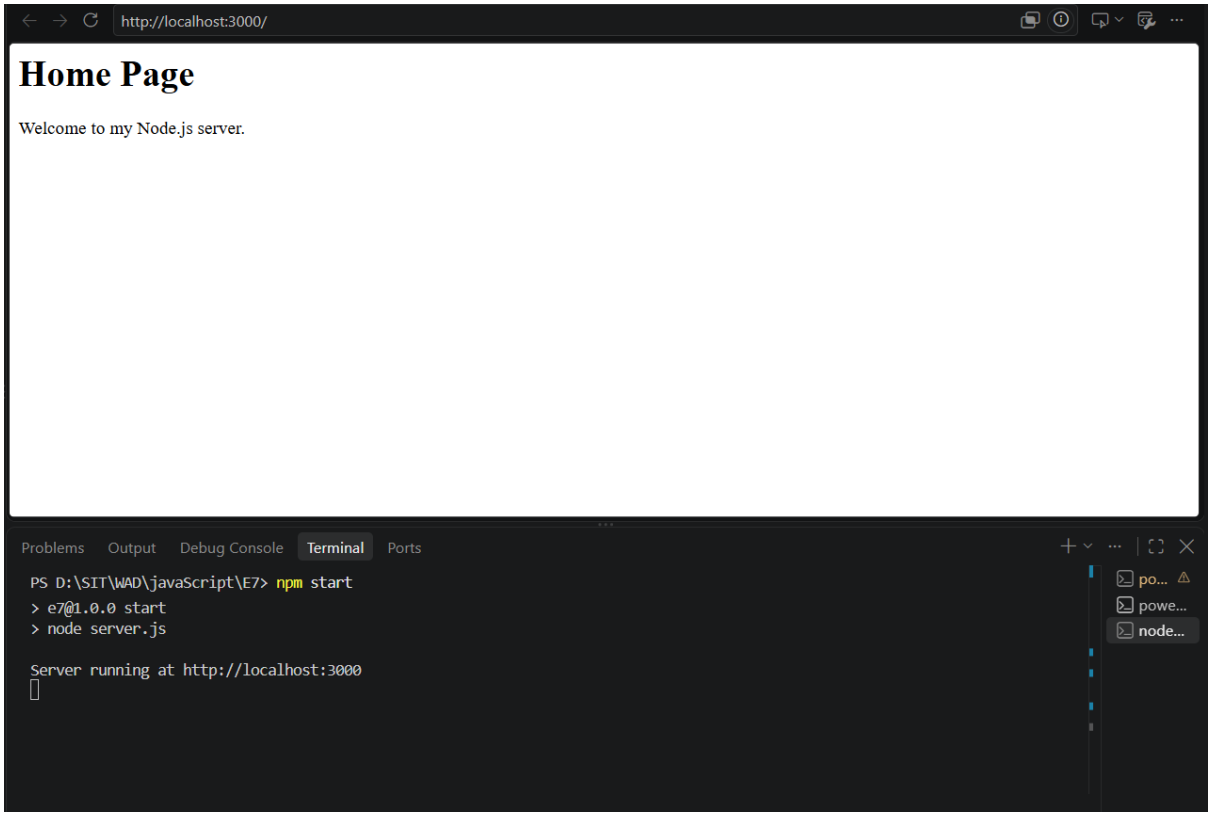
```
PS D:\SIT\WAD\javaScript\E7> node index.js
Platform: win32
CPU Cores: 8
2 + 3 = 5
4 squared = 16
PS D:\SIT\WAD\javaScript\E7>
```

Problems Output Debug Console Terminal Ports

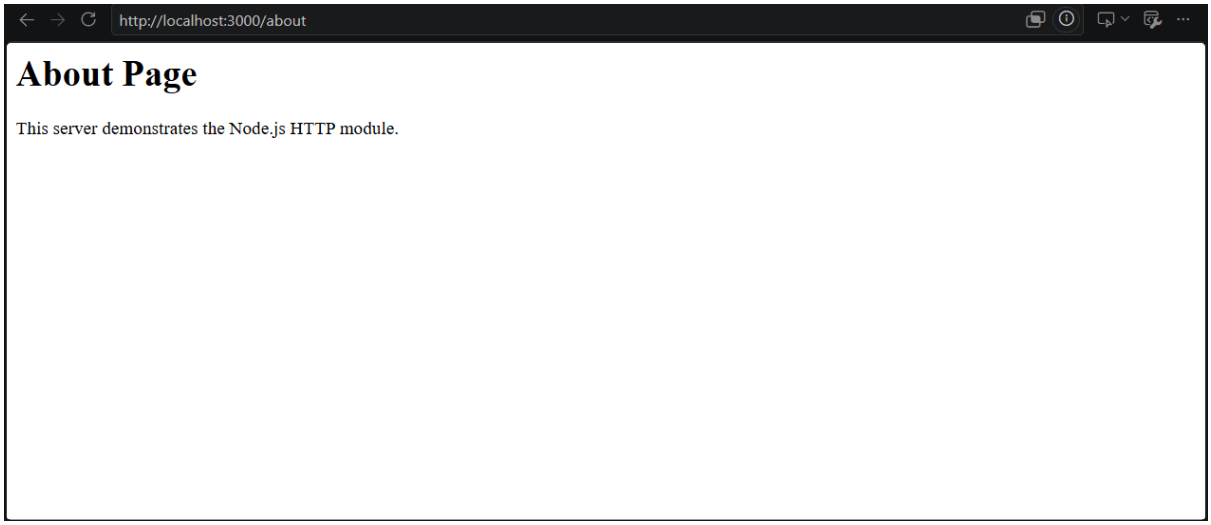
```
PS D:\SIT\WAD> cd javaScript
PS D:\SIT\WAD\javaScript> cd E7
PS D:\SIT\WAD\javaScript\E7> npm init -y
Wrote to D:\SIT\WAD\javaScript\E7\package.json:

{
  "name": "e7",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS D:\SIT\WAD\javaScript\E7>
```


http://localhost:3000/
Home Page
Welcome to my Node.js server.
Problems
Output
Debug Console
Terminal
Ports
PS D:\SIT\WAD\javaScript\E7> npm start
> e7@1.0.0 start
> node server.js
Server running at http://localhost:3000
po...
powe...
node...


http://localhost:3000/contact
Contact Page
Email: student@example.com





Hands-on Assignment

1. Create a local module named greetings.js exporting at least 3 functions (e.g. morningGreeting, eveningGreeting, formatName) and use them inside your HTTP server responses.
2. Extend the server to serve at least 5 distinct routes (Home, About, Contact, Services, 404) each returning a distinct styled HTML fragment (inline <style> is fine).
3. Add a /time route that responds with the current server date and time (use the built-in Date object), and a /random route that returns a random number between 1 and 100 on every request.
4. Use the os module to build a /server-info route that displays the platform, total memory, and free memory of the host machine in a formatted HTML table.

greetings.js

```
function morningGreeting() {
   return "Good Morning! Welcome to our Node.js application.";
}
```

```
function eveningGreeting() {
   return "Good Evening! Thank you for visiting our Node.js application.";
}

function formatName(firstName, lastName) {
   return `${firstName} ${lastName}`;
}

module.exports = {
   morningGreeting,
   eveningGreeting,
   formatName
};
```

hands_on_server.js

```
const http = require('http');
const os = require('os');
const greetings = require('./greetings');

const PORT = 3001;

const server = http.createServer((req, res) => {

   res.setHeader('Content-Type', 'text/html');

   // HOME ROUTE
   if (req.url === '/') {

      res.statusCode = 200;

      res.end(`
         <html>
         <head>
            <style>
               body {
                  font-family: Arial;
                  background: #e3f2fd;
                  text-align: center;
                  padding: 50px;
               }
```

```
            h1 {
                color: #1565c0;
            }
        </style>
    </head>

    <body>
        <h1>Home Page</h1>
        <p>${greetings.morningGreeting()}</p>
        <p>Welcome to our Node.js Web Application.</p>
    </body>
    </html>
  `);
}

// ABOUT ROUTE
else if (req.url === '/about') {

  res.statusCode = 200;

  res.end(`
    <html>
    <head>
        <style>
            body {
                font-family: Arial;
                background: #f3e5f5;
                text-align: center;
                padding: 50px;
            }

            h1 {
                color: #6a1b9a;
            }
        </style>
    </head>

    <body>
        <h1>About Page</h1>
        <p>${greetings.eveningGreeting()}</p>
        <p>This application demonstrates Node.js modules and the HTTP module.</p>
```

```
    </body>
    </html>
  `);
}

// CONTACT ROUTE
else if (req.url === '/contact') {

  res.statusCode = 200;

  res.end(`
    <html>
    <head>
      <style>
        body {
          font-family: Arial;
          background: #e8f5e9;
          text-align: center;
          padding: 50px;
        }

        h1 {
          color: #2e7d32;
        }
      </style>
    </head>

    <body>
      <h1>Contact Page</h1>
      <p>Contact Person: ${greetings.formatName('Vatsal', 'Student')}</p>
      <p>Email: student@example.com</p>
    </body>
    </html>
  `);
}

// SERVICES ROUTE
else if (req.url === '/services') {

  res.statusCode = 200;
```

```
    res.end(`
      <html>
      <head>
        <style>
          body {
            font-family: Arial;
            background: #fff3e0;
            text-align: center;
            padding: 50px;
          }

          h1 {
            color: #ef6c00;
          }

          li {
            margin: 10px;
          }
        </style>
      </head>

      <body>
        <h1>Services Page</h1>

        <ul>
          <li>Web Application Development</li>
          <li>Node.js Backend Development</li>
          <li>HTTP Server Development</li>
          <li>API Development</li>
        </ul>
      </body>
      </html>
    `);
}

// TIME ROUTE
else if (req.url === '/time') {

    const currentDateTime = new Date();

    res.statusCode = 200;
```

```
  res.end(`
    <html>
    <head>
      <style>
        body {
          font-family: Arial;
          background: #ede7f6;
          text-align: center;
          padding: 50px;
        }

        h1 {
          color: #4527a0;
        }

        .time {
          font-size: 24px;
          font-weight: bold;
        }
      </style>
    </head>

    <body>
      <h1>Server Date and Time</h1>
      <p class="time">${currentDateTime}</p>
    </body>
    </html>
  `);
}

// RANDOM NUMBER ROUTE
else if (req.url === '/random') {

  const randomNumber = Math.floor(Math.random() * 100) + 1;

  res.statusCode = 200;

  res.end(`
    <html>
    <head>
```

```
        <style>
          body {
            font-family: Arial;
            background: #fce4ec;
            text-align: center;
            padding: 50px;
          }

          h1 {
            color: #c2185b;
          }

          .number {
            font-size: 50px;
            font-weight: bold;
          }
        </style>
      </head>

      <body>
        <h1>Random Number</h1>
        <p class="number">${randomNumber}</p>
        <p>Random number between 1 and 100.</p>
      </body>
      </html>
    `);
}

// SERVER INFO ROUTE
else if (req.url === '/server-info') {

    const platform = os.platform();
    const totalMemory = os.totalmem();
    const freeMemory = os.freemem();

    res.statusCode = 200;

    res.end(`
      <html>
      <head>
        <style>
```

```
      body {
        font-family: Arial;
        background: #f5f5f5;
        padding: 50px;
      }

      h1 {
        text-align: center;
        color: #212121;
      }

      table {
        width: 70%;
        margin: auto;
        border-collapse: collapse;
        background: white;
      }

      th, td {
        border: 1px solid #999;
        padding: 15px;
        text-align: left;
      }

      th {
        background: #424242;
        color: white;
      }
    </style>
</head>

<body>

    <h1>Server Information</h1>

    <table>
      <tr>
        <th>Information</th>
        <th>Details</th>
      </tr>
```

```
            <tr>
              <td>Platform</td>
              <td>${platform}</td>
            </tr>

            <tr>
              <td>Total Memory</td>
              <td>${(totalMemory / (1024 ** 3)).toFixed(2)} GB</td>
            </tr>

            <tr>
              <td>Free Memory</td>
              <td>${(freeMemory / (1024 ** 3)).toFixed(2)} GB</td>
            </tr>
          </table>

      </body>
      </html>
    `);
}

// 404 ROUTE
else {

  res.statusCode = 404;

  res.end(`
    <html>
    <head>
      <style>
        body {
          font-family: Arial;
          background: #ffebee;
          text-align: center;
          padding: 50px;
        }

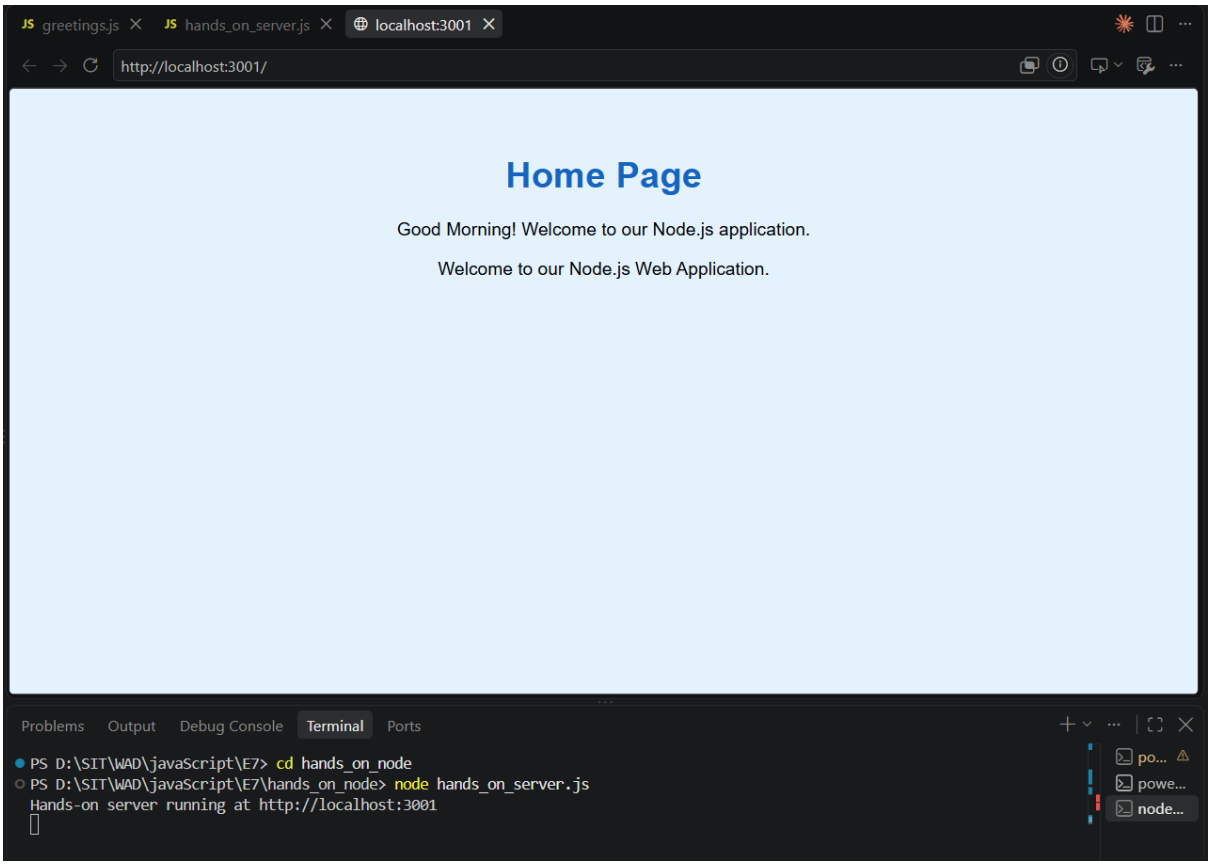
        h1 {
          color: #c62828;
        }
      </style>
```

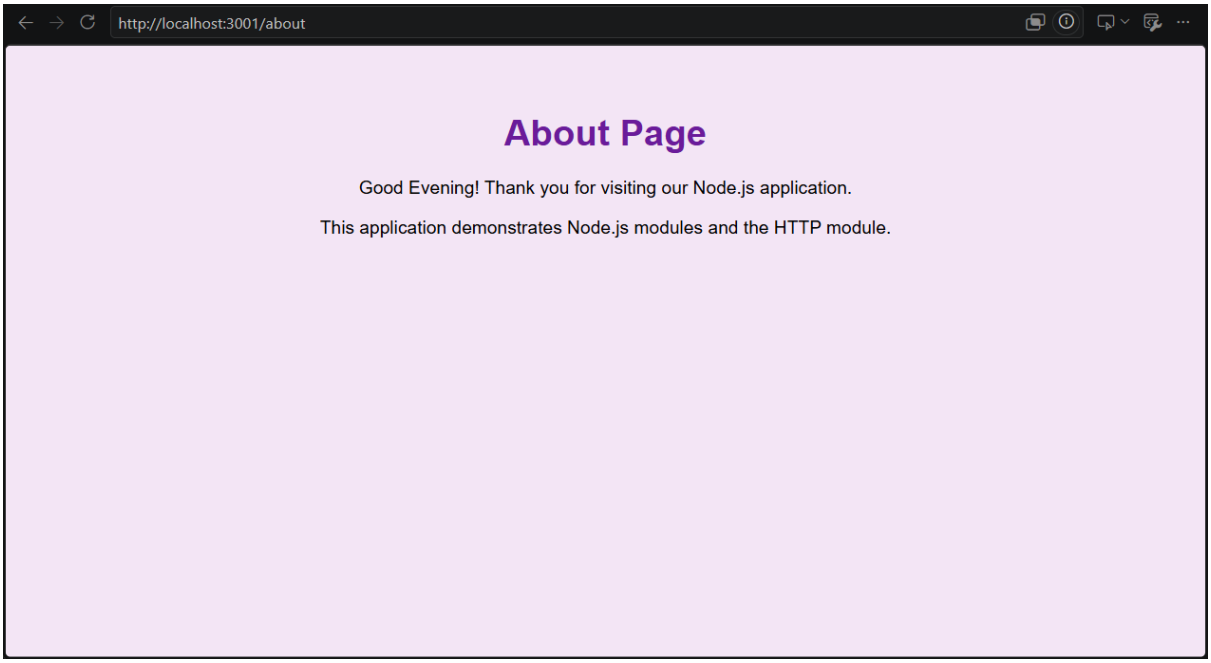
```
        </head>

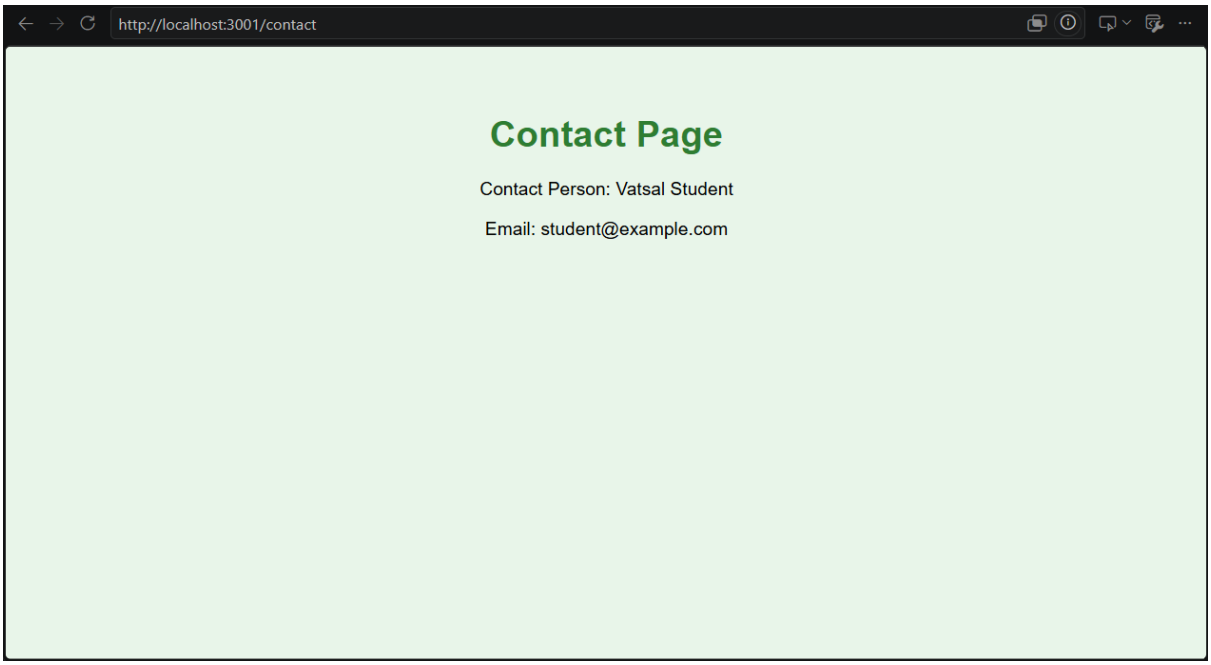
        <body>
          <h1>404 - Page Not Found</h1>
          <p>The requested route does not exist.</p>
        </body>
        </html>
      `);
  }
});

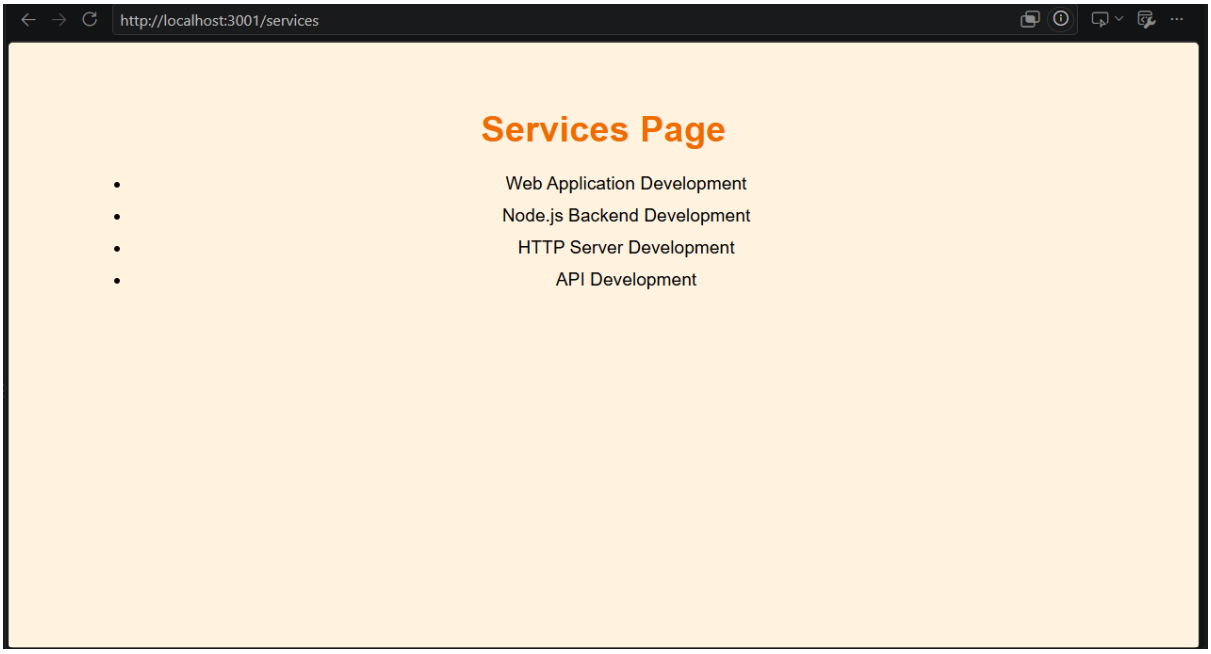
server.listen(PORT, () => {
  console.log(`Hands-on server running at http://localhost:${PORT}`);
});
```

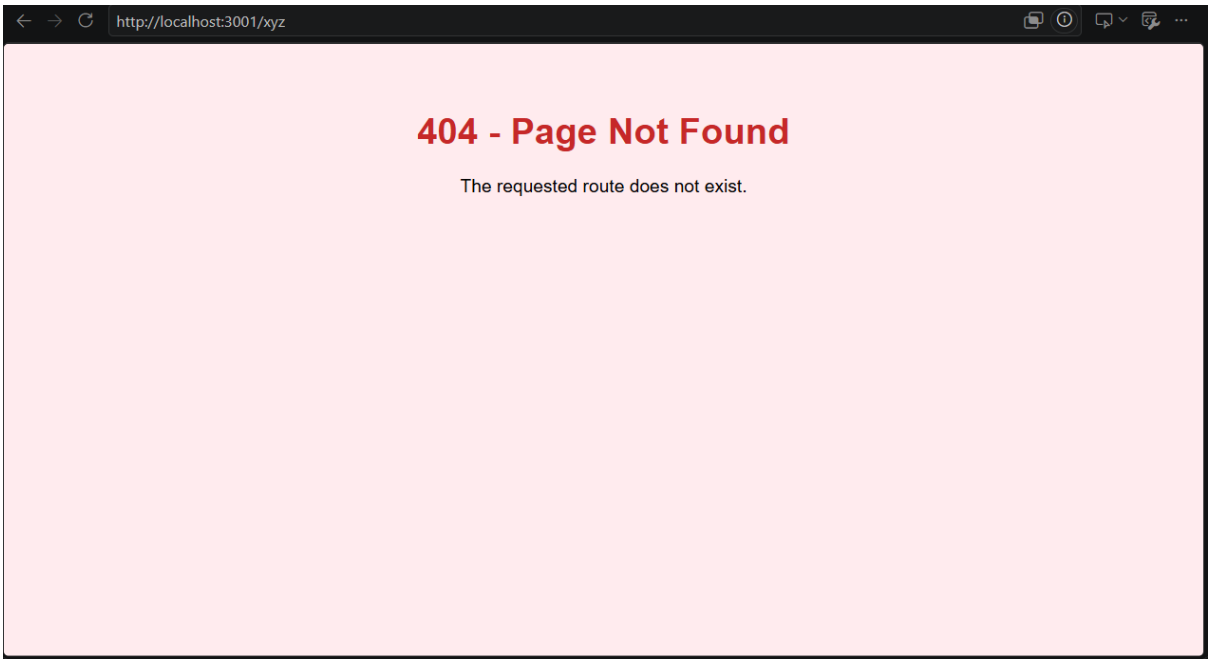
Output:

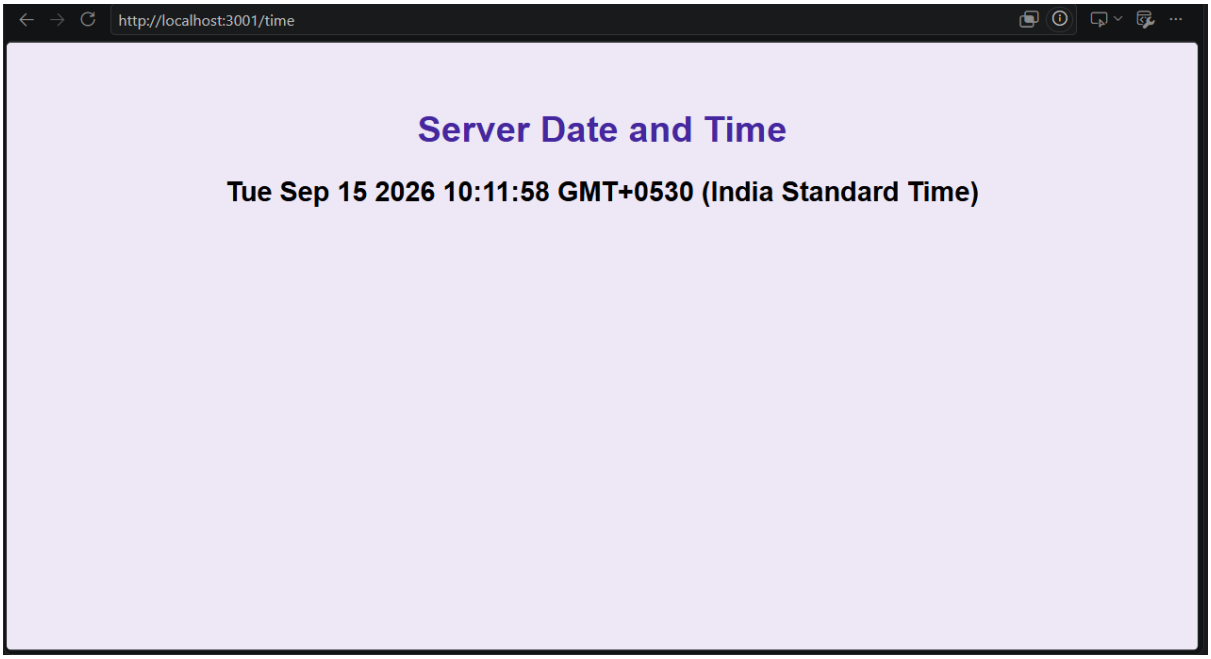


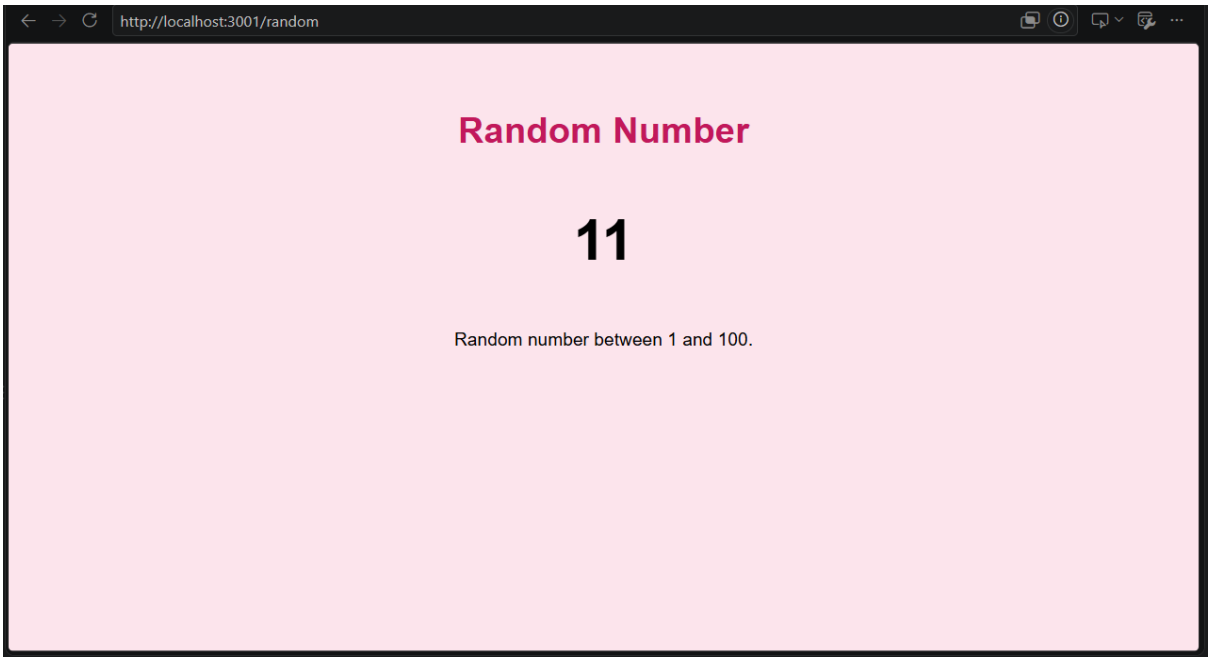

http://localhost:3001/about
About Page
Good Evening! Thank you for visiting our Node.js application.
This application demonstrates Node.js modules and the HTTP module.

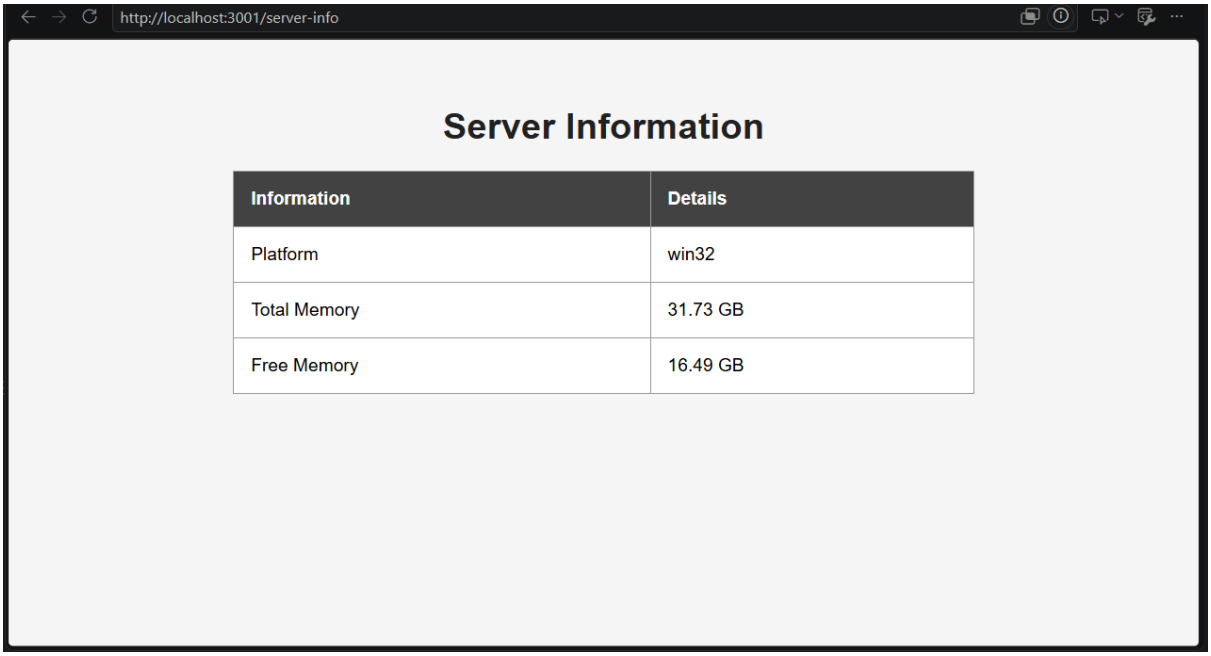

http://localhost:3001/contact
Contact Page
Contact Person: Vatsal Student
Email: student@example.com


http://localhost:3001/services
Services Page
Web Application Development
Node.js Backend Development
HTTP Server Development
API Development


http://localhost:3001/xyz
404 - Page Not Found
The requested route does not exist.


http://localhost:3001/time
Server Date and Time
Tue Sep 15 2026 10:11:58 GMT+0530 (India Standard Time)


http://localhost:3001/random
Random Number
11
Random number between 1 and 100.



Conclusion:

I learned how to create a backend web application using Node.js and the built-in HTTP module. I also learned about local modules, routing, HTTP methods, server information using the `os` module, and dynamic responses using date and random numbers.